upGrad

Pandas DataFrames

Key Definitions & Concepts

`DataFrame`: A two-dimensional, size-mutable, labelled data structure with columns of potentially different types.

Indexing: Refers to selecting rows and columns using position (`iloc`) or labels (`loc`).

Fancy Indexing: Supplying a list or array of labels/positions to select specific rows or columns.

Method Chaining: Writing successive operations on a DataFrame in one statement.

Grouping: The act of splitting data into groups and applying an aggregation function.

Wrangling: The process of cleaning, transforming, and reshaping raw data into a usable format.

Core Ideas & Theoretical Intuition

- A DataFrame is like a **spreadsheet with labelled rows and columns**, built for high-performance operations.
- Think of `.iloc` as NumPy-style **position-based access**, and `.loc` as dictionary-style **label-based access**.
- Merging is analogous to SQL joins — critical for combining datasets from multiple sources.
- Grouping and pivoting allow **multilevel summarisation**, ideal for exploratory data analysis.
- Most DataFrame operations are **non-destructive**, enabling method chaining and functional transformations.

Implementation

Creating DataFrames

Python

```python
import pandas as pd

# From dictionary
data = {'Name': ['Alice', 'Bob'], 'Age': [25, 30]}
df = pd.DataFrame(data)

# From list of lists
df2 = pd.DataFrame([[1, 2], [3, 4]], columns=['A', 'B'])

# With custom index
pd.DataFrame(data, index=['a', 'b'])
```

Attributes

Python

```python
df.dtypes          # Data type of each column

df.ndim            # Number of dimensions

df.shape           # Rows × Columns

df.size            # Total elements

df.index           # Row index object

df.columns         # Column names

df.values          # Underlying NumPy array
```

upGrad

Info & Summary Functions

Python

```python
df.info()          # Structure summary

df.head(3)         # First 3 rows; similarly for df.tail(3)

df.describe()      # Stats for numeric columns
```

Reading Data (CSV Example)

Python

```python
pd.read_csv('data.csv', header=0, index_col='ID',

            usecols=['ID', 'Name', 'Score'],

            nrows=100, skiprows=1)
```

Indexing and Slicing

`iloc` (position-based)

Python

```python
df.iloc[0]                # First row

df.iloc[:, 1]             # All rows, second column

df.iloc[0:2, 1:3]         # Rows 0-1, Columns 1-2
```

`loc` (label-based)

```python
df.loc['a']               # Row with label 'a'

df.loc[:, 'Age']          # Entire 'Age' column

df.loc['a':'b', ['Name']] # Slice rows and specific col
```

Fancy Indexing

```python
# Rows with labels 'a' and 'c' and columns 'Name' and 'Age'

df.loc[['a', 'c'], ['Name', 'Age']]

# Rows at positions 0 and 2 and columns at positions 0 and 1

df.iloc[[0, 2], [0, 1]]
```

Operations on DataFrames

```python
df * 10                   # Multiply numeric values

df['Bonus'] = df['Age'] * 0.1  # Add new column


df.sum(), df.mean()       # Aggregations

df.sort_values('Age')     # Sort rows

df.apply(len)             # Apply function to columns
```

```
df['Name'].map(str.upper)  # Element-wise string map


# Method chaining

df.dropna().query("Age > 20").sort_values('Age')
```

Working with DataFrames

Python

```
df.columns = df.columns.str.lower()       # Rename cols

df.rename(columns={'age': 'year'})         # Rename

df.fillna(0)                               # Fill missing
```

Merging DataFrames

Python

```
pd.merge(df1, df2, how='inner', on='ID')

# SQL-style merge

pd.concat([df1, df2], axis=0)

# Stack rows
```

- how= options: 'inner', 'left', 'right', 'outer'
- on= specifies column key; axis=1 for column-wise merge

Analysing DataFrames

Python

```
df.reset_index()                    # Move index to column

df['Score'].quantile(0.75)         # 75th percentile
```

Grouping and Aggregation

Python

```
df.groupby('Department')['Salary'].sum()

df.groupby(['Gender','Department']).agg({'Age': 'mean', 'Salary':
'max'})
# Group the DataFrame by both 'Gender' and 'Department',
# then compute the mean age and max salary for each group
```

Pivoting

Python

```
df.pivot(index='Date', columns='Region', values='Sales')


## Sample Output:

# Region      North  South

# Date

# 2023-01-01    100    150

# 2023-01-02    200    180
```

Wrangling a DataFrame

Python

```python
df.drop_duplicates()              # drop duplicate rows

df.dropna()                       # drop rows with null values

df['City'] = df['City'].str.strip().str.title()

df['Date'] = pd.to_datetime(df['Date'])
```

Summary

Task	Example Code	Output Description
Row by label	`df.loc['a']`	Row with index label 'a'
Col by position	`df.iloc[:, 1]`	Second column
Filter	`df[df['Age'] > 25]`	Rows with Age > 25
Merge	`pd.merge(df1, df2, on='ID')`	Combine tables
Groupby	`df.groupby('Team')['Score'].mean()`	Aggregated stats
Pivot	`df.pivot(index, columns, values)`	Reshaped table

Pitfalls and Misconceptions

- `.iloc` vs `.loc`: Position vs Label: using the wrong one may raise errors
- Modifying views: `.loc` may return a view, not a copy: use `.copy()` if needed
- `map()` vs `apply()`: `map` works on Series elements; `apply` can be column-wise on DataFrame
- Be cautious when chaining inplace operations (e.g., `df.dropna(inplace=True)` followed by `df.sort_values()`)

Common Interview Questions

1. What are the key differences between a NumPy array and a Python list?

Topic: NumPy Basics
Expected Points:

- Fixed-size, homogeneous vs dynamic, heterogeneous
- Vectorised operations in NumPy
- Memory efficiency and performance benefits

2. Explain broadcasting in NumPy. Give an example of a valid and an invalid broadcast operation.

Topic: NumPy Broadcasting
Expected Points:

- Rules for shape compatibility
- Example: `(3, 1)` + `(1, 4)` → `(3, 4)`
- Invalid example where trailing dimensions don't match

3. What does `.reshape()` do in NumPy? How is it different from `.ravel()` and `.flatten()`?

Topic: Array Reshaping & Views
Expected Points:

- `reshape` changes shape without changing data
- `flatten` returns a copy; `ravel` returns a view (if possible)

4. How would you remove duplicate values from a Pandas Series or DataFrame?

Topic: Data Cleaning in Pandas
Expected Points:

- `Series.drop_duplicates()`
- `DataFrame.drop_duplicates(subset=['col1', ...])`

5. How are `.loc[]` and `.iloc[]` different? When would you use each?

Topic: Pandas Indexing
Expected Points:

- `.loc[]` uses label-based indexing
- `.iloc[]` uses integer position indexing
- Edge cases (e.g., mixed-type index)

6. What is method chaining in Pandas? Why is it useful?

Topic: Code Style and Efficiency
Expected Points:

- Writing operations like: `df.dropna().query(...).sort_values(...)`
- Benefits: readable, compact, avoids intermediate variables

7. Explain the difference between `map()`, `apply()`, and `applymap()` in Pandas.

Topic: Functional Programming with Pandas
Expected Points:

- `map()` → element-wise on Series
- `apply()` → row/column-wise on DataFrame
- `applymap()` → element-wise on DataFrame

8. Describe how you would merge two DataFrames in Pandas. What types of joins are supported?

Topic: Combining DataFrames
Expected Points:

- `pd.merge(left, right, how='inner', on='key')`
- `how=` supports: 'inner', 'left', 'right', 'outer'
- Can also join on the index or use `concat()` for stacking

9. You read a CSV file and find some columns are not needed, and the headers are in the second row. How would you load only the relevant data?

Topic: Data Import & Preprocessing
Expected Points:

- Use `usecols=`, `skiprows=`, `header=` in `read_csv()`

10. How would you calculate the average sales per product category in a DataFrame? How would you reshape the result into a pivot table?

Topic: Grouping and Pivoting
Expected Points:

- `df.groupby('Category')['Sales'].mean()`
- `df.pivot(index='Date', columns='Category', values='Sales')`

upGrad

Lecture Notes:
Matplotlib and Seaborn

Key Definitions & Concepts

- **Matplotlib (plt)**: A low-level Python library that gives fine control over every plot element
- **Seaborn (sns)**: A high-level wrapper over Matplotlib designed for **statistical visualisation**
- **Figure**: The overall plot canvas
- **Axes**: Subplots or coordinate spaces within the figure
- **Tick Labels**: Numbers or categories on x- and y-axes
- **Hues**: Used in Seaborn to distinguish data by categories using colour
- **Matplotlib** gives precise control over plots, while **Seaborn** provides pre-built, declarative interfaces for clean visualisations
- Plot customisation, like `xticks`, `ylabel`, and `legend,` help in **communicating insights** clearly
- `hue` in Seaborn maps categories to colours, allowing **groupwise comparison**
- **Boxplots** summarise distributions using **median, IQR, and whiskers**; **histograms** show distributions using **bins**

upGrad

Implementation

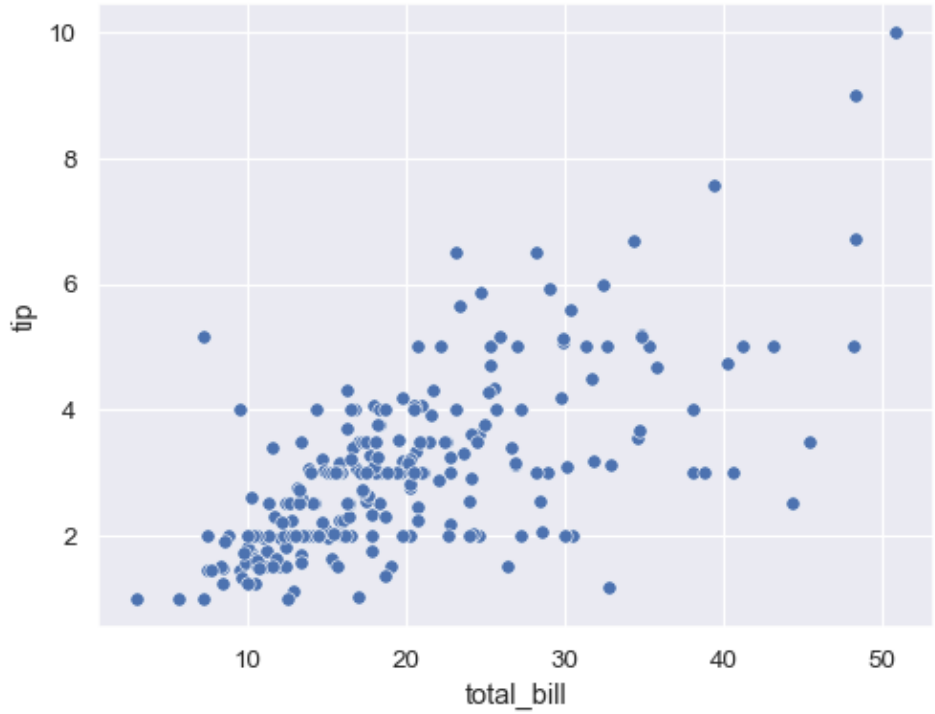
Plot Definitions in Matplotlib vs Seaborn

Task	Matplotlib (`plt`)	Seaborn (`sns`)
Create Plot	`plt.plot(x, y)` Basic line plot, minimal defaults; requires manual formatting	`sns.lineplot(x=x, y=y, data=df)` Styled by default; Works seamlessly with DataFrames
Set Title	`plt.title("Title")` Sets the title on the current figure	`ax.set_title("Title")` Used with `sns` plots inside subplots; or `plt.title()`
Axis Labels	`plt.xlabel("X")`, `plt.ylabel("Y")` Full manual control	Same as `plt`; Seaborn does not override axis label behaviour
Legend	`plt.legend()` Manual labels required (via `label=` in plot calls)	Automatically generates from `hue` or `label=`; Use `plt.legend()` to customise
Tick Labels	`plt.xticks()`, `plt.yticks()` Precise tick control (values, labels, rotation)	Inherits from Matplotlib; commonly used with `plt.xticks(rotation=45)`
Save Plot	`plt.savefig("figure.png")` Saves the current figure as an image	Same; Seaborn uses Matplotlib's backend for rendering and saving
Hue (Colour Grouping)	Manual Use `color=`, `label=` for each group or series manually	Automatic Use `hue='column'` to group and colour data by a categorical variable
Styling & Themes	Requires manual setting: colours, grid, font, etc. via `plt.style.use()`	Comes with built-in themes (`sns.set_theme()`); Consistent aesthetics out-of-the-box

upGrad

Plot Types

Scatter Plot



Matplotlib

Python

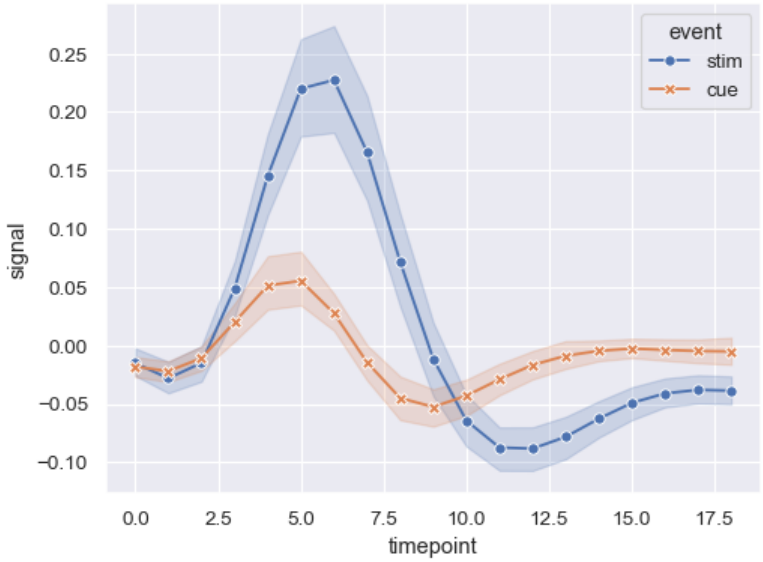
```
plt.scatter(x='Age', y='Salary', data=df, color='blue',
label='Employees')
plt.xlabel('Age')
plt.ylabel('Salary')
plt.legend()
```

Seaborn

Python

```
sns.scatterplot(x='Age', y='Salary', data=df, hue='Department')
```

Line Plot



Matplotlib

Python

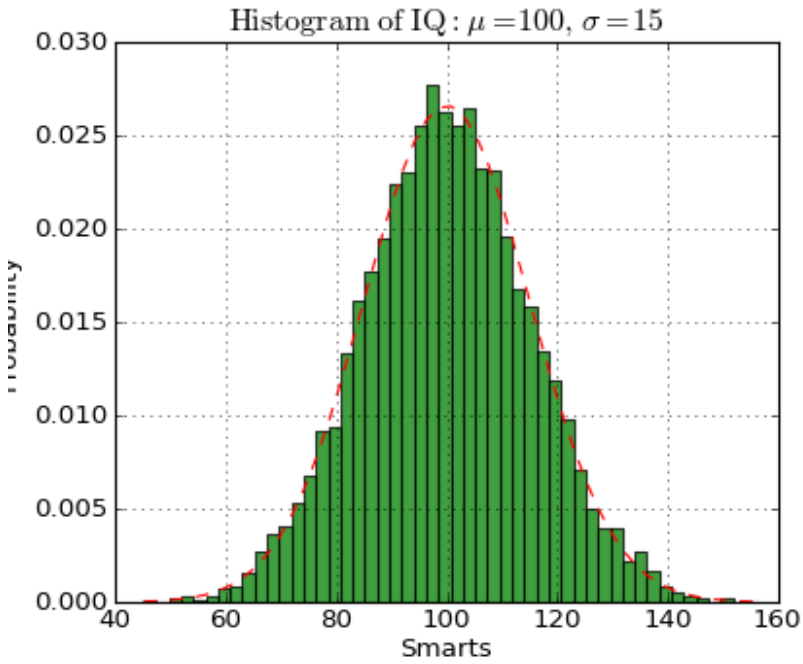
```
plt.plot(df['Month'], df['Sales'], label='Sales')
plt.xticks(rotation=45)
plt.xlabel('Month')
plt.ylabel('Sales')
plt.legend()
```

Seaborn

Python

```
sns.lineplot(x='Month', y='Sales', data=df, hue='Region')
```

Histogram



Matplotlib

Python

```
plt.hist(df['Marks'], bins=10, color='skyblue',
edgecolor='black')
```

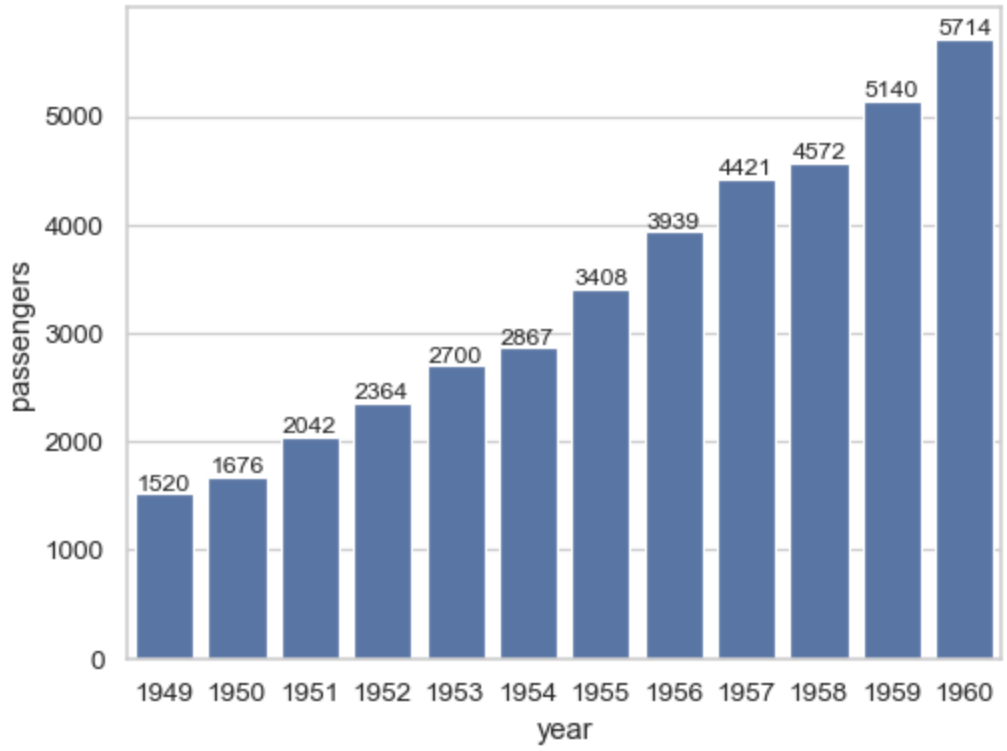
Seaborn

Python

```
sns.histplot(df['Marks'], bins=10, kde=True)
```

`bins` define the granularity of distribution; `kde=True` overlays a smooth curve.

Bar Plot



Matplotlib

Python

```
categories = df['Department'].value_counts()

plt.bar(categories.index, categories.values)
```

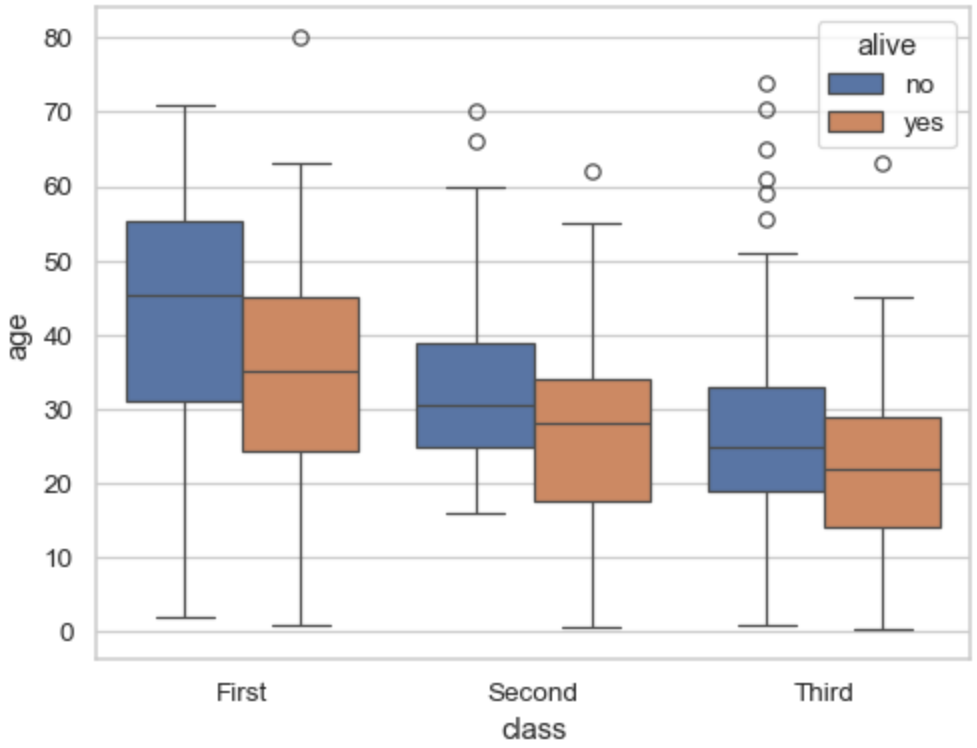
Seaborn

Python

```
sns.barplot(x='Department', y='Salary', data=df,
estimator='mean')
```

Seaborn allows aggregation (`sum`, `mean`, `count`) directly.

Box Plot



Seaborn

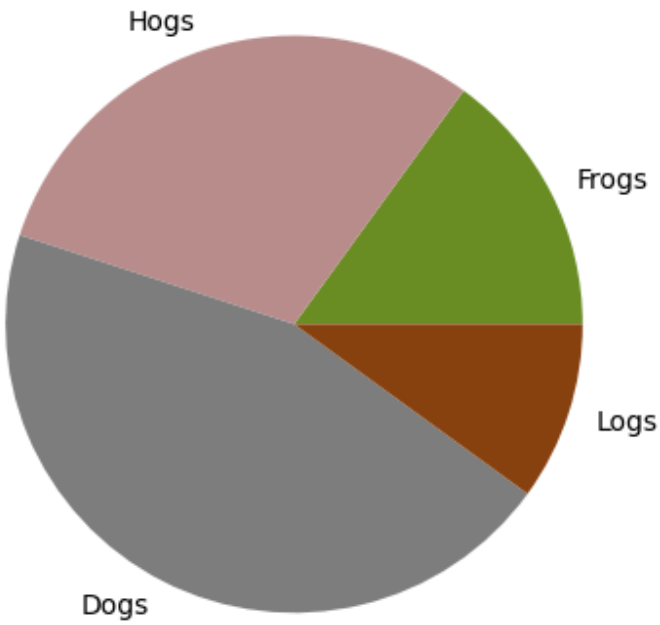
Python

```
sns.boxplot(x='Department', y='Salary', data=df)
```

Interpretation:

- **Median**: Central line in the box
- **IQR**: Height of the box (Q3 - Q1)
- **Whiskers**: Extend to 1.5×IQR
- **Outliers**: Points outside the whiskers

Pie Chart (Matplotlib only)



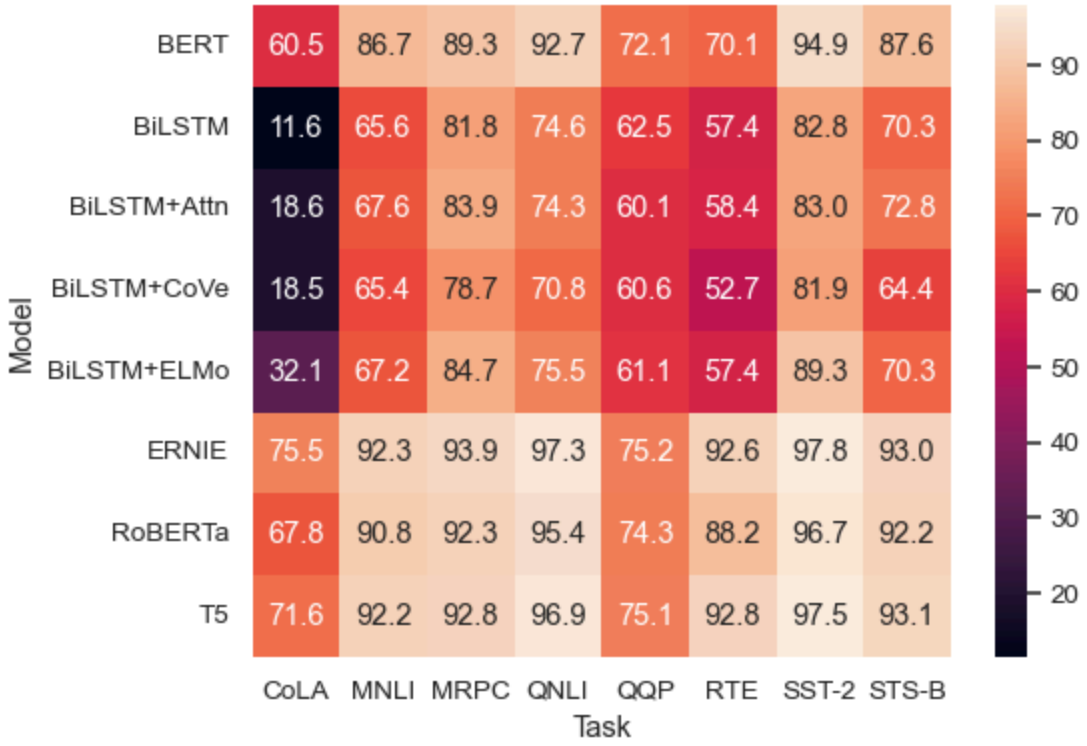

```python
sizes = [40, 30, 20, 10]
labels = ['A', 'B', 'C', 'D']
explode = [0.1, 0, 0, 0]
plt.pie(sizes, labels=labels, explode=explode, autopct='%1.1f%%')
```

We could add annotations to show the percentage of each slice of the pie chart using `autopct` as well as explode a particular slice for emphasis using `explode`.

`explode` takes in a sequence where each element corresponds to a slice in your pie chart

- A value of 0 at an index means that the corresponding slice will not be exploded.
- A value greater than 0 (e.g., 0.1, 0.2) at an index will explode that slice by the specified fraction of the radius. A larger value will result in a greater separation.

Heatmap



Seaborn

Python

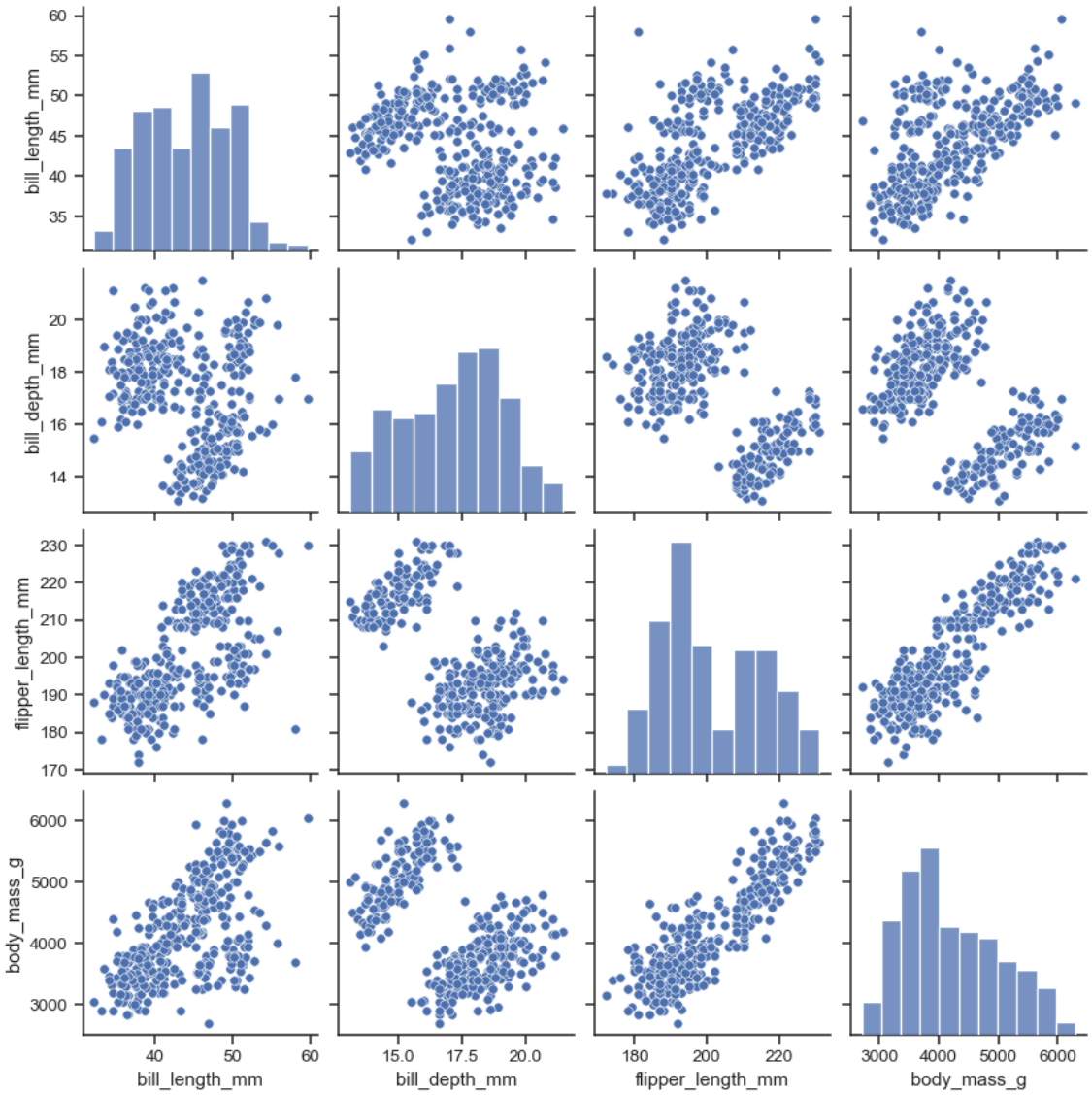
```
corr = df.corr()

sns.heatmap(corr, annot=True, cmap='coolwarm')
```

Common use: show a **correlation matrix** for numeric variables.

upGrad

Pairplot



Seaborn

Python

```
sns.pairplot(df, hue='Species')
```

Shows pairwise scatter plots + histograms for all numerical columns. Excellent for multivariate inspection.

upGrad

Summary

Plot Type	Use Case	Key Argument(s)
`scatterplot`	Relationship between two variables	`x`, `y`, `hue`, `style`
`lineplot`	Trends over ordered data (e.g., time)	`x`, `y`, `hue`, `ci`
`histplot`	Distribution of a numeric variable	`bins`, `kde`, `hue`
`barplot`	Categorical mean/counts	`estimator`, `ci`, `hue`
`boxplot`	Distribution + outliers	`x`, `y`, `hue`
`pie`	Part-to-whole (not in Seaborn)	`labels`, `autopct`, `explode`
`heatmap`	Matrix/Correlation visualisation	`annot`, `cmap`, `vmin`, `vmax`
`pairplot`	Explore multivariate numeric distributions	`hue`, `palette`, `kind`

Pitfalls

- `hue` only works in **Seaborn** (not Matplotlib)
- Too many categories in pie/bar → cluttered visuals
- Histogram interpretation depends on **bin size**
- Pairplots can be slow on large datasets; sample first
- Misaligned axis labels or missing legends reduce readability

Additional Reading

- Seaborn Documentation
- Matplotlib Documentation
- Seaborn Examples Gallery